Batch Normalization

Finding the Scale and Shift Parameters

Moin Mostakim

Department of Computer Science and Engineering
Brac University

June 2026

Outline

1. Motivation
2. Forward pass
3. The math problem
4. Worked example
5. Training vs inference
6. Takeaways

Why normalize activations?

Problem: internal covariate shift

- Layer inputs change as earlier layers update.
- Large activations $\Rightarrow$ saturated gradients (sigmoid/tanh).
- Training slows; learning rate must stay small.

Idea (Ioffe & Szegedy, 2015): per feature, force

$$\text{mean} \approx 0, \quad \text{variance} \approx 1$$

inside each mini-batch, then learn how much to rescale and shift.

Two learnable knobs per feature

- **Scale** γ — stretch or compress
- **Shift** β — move up or down

Key insight

Pure normalization would limit what the network can represent. γ and β restore expressiveness while keeping training stable.

Notation: one feature in a mini-batch

Mini-batch of size m. For feature j, collect activations

$$\mathcal{B} = \{x_1, x_2, \ldots, x_m\}.$$

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{(batch mean)}$$

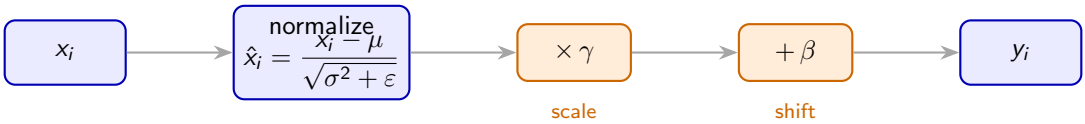
$$\sigma_{\mathcal{B}}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{(batch variance)}$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} \qquad \text{(normalized)}$$

$$y_i = \gamma\,\hat{x}_i + \beta \qquad \text{(scale \& shift output)}$$

$\varepsilon > 0$ avoids division by zero. **One pair (γ_j, β_j) per feature j.**

Pipeline: normalize, then scale and shift



Compact form

$$y_i = \gamma \left(\frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} \right) + \beta$$

Special cases for γ and β

- If $\gamma = 1$ and $\beta = 0$: output is pure standardization.
- If $\gamma = \sigma_{\mathcal{B}}$ and $\beta = \mu_{\mathcal{B}}$:

$$y_i = \gamma \hat{x}_i + \beta = \sigma_{\mathcal{B}} \cdot \frac{x_i - \mu_{\mathcal{B}}}{\sigma_{\mathcal{B}}} + \mu_{\mathcal{B}} = x_i.$$

  The layer can **undo** normalization and recover the original activations.
- During training, SGD finds γ, β that minimize the loss jointly with all other weights.

Problem statement

Given

- Normalized values $\hat{x}_1, \ldots, \hat{x}_m$ (already computed from the batch)
- Upstream gradient $\dfrac{\partial L}{\partial y_i}$ for each sample i
- Current parameters γ, β

Find

1. **Scale update:** $\dfrac{\partial L}{\partial \gamma}$
2. **Shift update:** $\dfrac{\partial L}{\partial \beta}$
3. **Downstream gradient:** $\dfrac{\partial L}{\partial \hat{x}_i}$ (to backprop through normalization)

These drive gradient descent: $\gamma \leftarrow \gamma - \eta\, \partial L/\partial \gamma$, $\beta \leftarrow \beta - \eta\, \partial L/\partial \beta$.

Finding the shift β

Forward: $y_i = \gamma\,\hat{x}_i + \beta$.

By the chain rule, β affects every y_i in the batch equally:

$$\frac{\partial L}{\partial \beta} = \sum_{i=1}^{m} \frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial \beta} = \sum_{i=1}^{m} \frac{\partial L}{\partial y_i} \cdot 1 = \boxed{\sum_{i=1}^{m} \frac{\partial L}{\partial y_i}}.$$

Interpretation: β is a bias. Its gradient is the *sum* of upstream errors — exactly like a bias in a linear layer.

Finding the scale γ

Again from $y_i = \gamma\,\hat{x}_i + \beta$:

$$\frac{\partial L}{\partial \gamma} = \sum_{i=1}^{m} \frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial \gamma} = \sum_{i=1}^{m} \frac{\partial L}{\partial y_i} \cdot \hat{x}_i = \boxed{\sum_{i=1}^{m} \frac{\partial L}{\partial y_i}\,\hat{x}_i}.$$

Interpretation: γ scales each normalized activation. Its gradient is a *weighted sum* of upstream errors, weights $= \hat{x}_i$.

Also needed for backprop into earlier layers:

$$\frac{\partial L}{\partial \hat{x}_i} = \frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial \hat{x}_i} = \boxed{\frac{\partial L}{\partial y_i}\,\gamma}.$$

Summary: scale & shift gradients

Parameter	Role	Gradient
β	shift (bias)	$\sum_{i=1}^{m} \frac{\partial L}{\partial y_i}$
γ	scale (weight)	$\sum_{i=1}^{m} \frac{\partial L}{\partial y_i} \hat{x}_i$
$\hat{x}_i$	normalized input	$\frac{\partial L}{\partial y_i} \gamma$

The harder part is $\partial L / \partial x_i$ (through $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}$), but γ and β themselves have the simple forms above.

Practice problem: setup

Mini-batch ($m = 4$) for one feature:

$$x = [2,\ 4,\ 6,\ 8].$$

Use $\varepsilon = 0$. Upstream gradients (from the loss):

$$\frac{\partial L}{\partial y} = [0.5,\ -0.5,\ 1.0,\ -1.0].$$

Current parameters: $\gamma = 1.2$, $\beta = 0.3$.

Tasks:

1. Compute $\mu_{\mathcal{B}}$, $\sigma^2_{\mathcal{B}}$, and each $\hat{x}_i$.
2. Compute $\partial L/\partial \beta$ and $\partial L/\partial \gamma$.
3. Compute each $\partial L/\partial \hat{x}_i$.

Step 1: normalize the batch

$$\mu_{\mathcal{B}} = \frac{2+4+6+8}{4} = 5, \qquad \sigma^2_{\mathcal{B}} = \frac{(-3)^2+(-1)^2+1^2+3^2}{4} = \frac{20}{4} = 5.$$

$$\hat{x}_i = \frac{x_i - 5}{\sqrt{5}} : \qquad \hat{x} = \left[-\frac{3}{\sqrt{5}}, -\frac{1}{\sqrt{5}}, \frac{1}{\sqrt{5}}, \frac{3}{\sqrt{5}}\right].$$

Numerically: $\hat{x} \approx [-1.342,\ -0.447,\ 0.447,\ 1.342]$.

Step 2: find shift β and scale γ gradients

Shift:

$$\frac{\partial L}{\partial \beta} = 0.5 + (-0.5) + 1.0 + (-1.0) = \boxed{0}.$$

Scale:

$$\begin{aligned}
\frac{\partial L}{\partial \gamma} &= 0.5\,\hat{x}_1 + (-0.5)\,\hat{x}_2 + 1.0\,\hat{x}_3 + (-1.0)\,\hat{x}_4 \\
&= \frac{1}{\sqrt{5}}\big[0.5(-3) + (-0.5)(-1) + 1.0(1) + (-1.0)(3)\big] \\
&= \frac{1}{\sqrt{5}}(-1.5 + 0.5 + 1.0 - 3.0) = \boxed{-\frac{3}{\sqrt{5}} \approx -1.342}.
\end{aligned}$$

With learning rate $\eta = 0.1$: $\gamma \leftarrow 1.2 - 0.1(-1.342) \approx 1.334$, $\beta \leftarrow 0.3 - 0.1(0) = 0.3$.

Step 3: gradient w.r.t. normalized activations

$$\frac{\partial L}{\partial \hat{x}_i} = \frac{\partial L}{\partial y_i}\,\gamma = 1.2 \cdot \frac{\partial L}{\partial y_i}.$$

$$\frac{\partial L}{\partial \hat{x}} = 1.2 \times [0.5,\ -0.5,\ 1.0,\ -1.0] = \boxed{[0.6,\ -0.6,\ 1.2,\ -1.2]}.$$

These values are passed backward into the normalization step to compute $\partial L/\partial x_i$ (and then into earlier layers).

Check: forward output with current γ, β

$$y_i = \gamma\,\hat{x}_i + \beta = 1.2\,\hat{x}_i + 0.3.$$

Using $\hat{x} \approx [-1.342,\ -0.447,\ 0.447,\ 1.342]$:

$$y \approx [-1.31,\ -0.24,\ 0.84,\ 1.91].$$

Normalization centers and unit-scales; γ, β map back to a useful range for the next layer.

Training vs inference

Training

- $\mu_{\mathcal{B}}$, $\sigma^2_{\mathcal{B}}$ from *current* mini-batch.
- γ, β updated every step.
- Running averages stored:

$$\text{EMA}_{\mu} \leftarrow (1-\alpha)\text{EMA}_{\mu} + \alpha\mu_{\mathcal{B}}$$

Inference

- Use population statistics EMA_{μ}, EMA_{σ^2} (not batch stats).
- γ, β fixed (learned values).

$$y = \gamma \frac{x - \text{EMA}_{\mu}}{\sqrt{\text{EMA}_{\sigma^2} + \varepsilon}} + \beta$$

At inference, scale and shift are still γ and β; only the mean/variance source changes.

Takeaways

1. Batch norm: **normalize** $\rightarrow$ **scale** (γ) $\rightarrow$ **shift** (β).
2. β gradient $=$ sum of upstream errors (like a bias).
3. γ gradient $=$ dot product of upstream errors with $\hat{x}$ (like a weight).
4. Setting $\gamma = 1, \beta = 0$ gives standardization; learning γ, β restores representational power.
5. The network can recover x_i exactly if $\gamma = \sigma_{\mathcal{B}}$, $\beta = \mu_{\mathcal{B}}$.

Reference

S. Ioffe & S. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML 2015.

Quick quiz

1. Why not stop after normalization? Why add γ and β?
2. If all $\partial L/\partial y_i = c$ (same constant), what is $\partial L/\partial \beta$? What about $\partial L/\partial \gamma$?
3. If $\gamma = 0$, what happens to y_i? What does $\partial L/\partial \hat{x}_i$ become?

Quick quiz

1. Why not stop after normalization? Why add γ and β?
2. If all $\partial L/\partial y_i = c$ (same constant), what is $\partial L/\partial \beta$? What about $\partial L/\partial \gamma$?
3. If $\gamma = 0$, what happens to y_i? What does $\partial L/\partial \hat{x}_i$ become?

Answers: (1) To allow the layer to undo normalization and learn the best scale/shift. (2) $\partial L/\partial \beta = mc$; $\partial L/\partial \gamma = c \sum_i \hat{x}_i = 0$ (normalized batch sums to 0 in $\hat{x}$ when centered). (3) $y_i = \beta$ (constant); $\partial L/\partial \hat{x}_i = 0$.